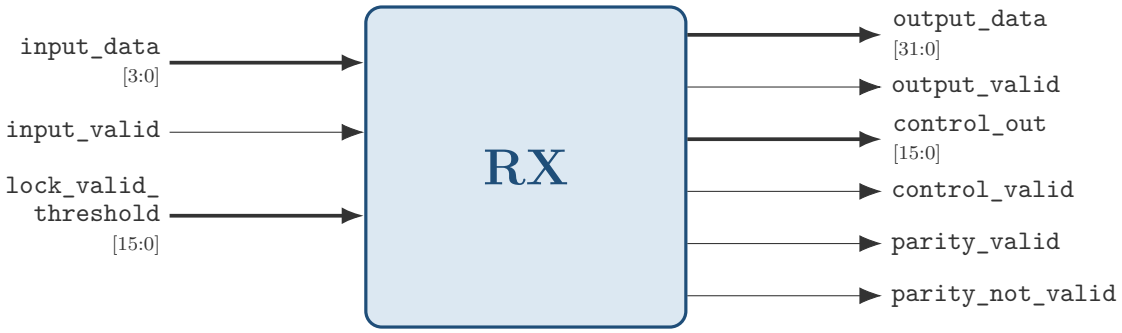
RX — Receiver Module

RTL Functional Specification



Document	RX RTL Specification
Block	DMA Receiver (RX)
Scope	RTL functional behavior only (no verification content)
Date	September 28, 2026

Contents

1 Overview **2**

2 Interface **2**
2.1 Block Diagram 2
2.2 Parameters 2
2.3 Ports 3

3 Packet Format **3**

4 Functional Behavior **3**
4.1 Phase Acquisition and Lock 3
4.2 Single-Bit Error Correction 4
4.3 Parity Validation 4
4.4 Output Control Gating 4

5 CRC Specification **4**
5.1 Data Window Format 4
5.2 Length CRC — CRC-16-CCITT 4
5.3 Control CRC — CRC-16-IBM/ANSI 5

6 Throughput Requirement **5**

7 System-Level Assumptions **5**

1 Overview

The **RX** block receives a continuous stream of packets on a narrow input bus and reconstructs the *Data* and *Control* payload that each packet carries. Because the packet boundary is not aligned with the input bus a priori, the module must first *discover* where packets begin (the *phase*), lock onto that phase once it is confident, and only then start emitting decoded data.

Once locked, the module tolerates a single-bit error in either the `LENGTH` field or the `CONTROL` field of a packet (each protected by its own 16-bit CRC) by correcting it in place, and it reports the parity health of every packet it outputs.

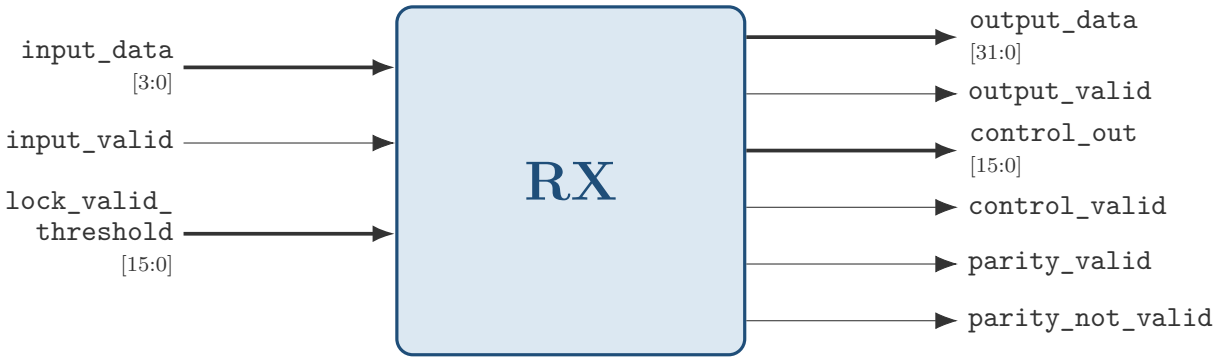
Key responsibilities

- Discover and lock the packet phase on an unaligned nibble stream.
- Extract `LENGTH`, `CONTROL` and `DATA` from every packet.
- Detect and correct single-bit errors in `LENGTH` and `CONTROL`.
- Validate end-to-end packet parity and flag the result.
- Suppress `CONTROL` output when its value is zero.

2 Interface

2.1 Block Diagram

Signals drawn with a thick arrow are multi-bit buses; signals drawn with a thin arrow are single-bit strobes / indications.



2.2 Parameters

Parameter	Default	Description / Legal range
`INPUT_DATA_WIDTH`	4	Width of the incoming data bus, in bits.
`LOCK_THRESHOLD_DATA_WIDTH`	16	Width of the lock-threshold bus, in bits.
`OUTPUT_DATA_WIDTH`	32	Width of the outgoing data bus, in bits.
`CONTROL_DATA_WIDTH`	16	Width of the outgoing control bus, in bits.

2.3 Ports

Signal	I/O	Width	Description
input_data	in	INPUT_DATA_WIDTH	Incoming packet stream.
input_valid	in	1	Qualifies input_data as valid on this cycle.
lock_valid_threshold	in	LOCK_THRESHOLD_DATA_WIDTH	Number of consecutive valid packets required, at the same phase, before the module locks.
output_data	out	OUTPUT_DATA_WIDTH	Decoded DATA payload of the current packet.
output_valid	out	1	Qualifies output_data as valid on this cycle.
control_out	out	CONTROL_DATA_WIDTH	Decoded CONTROL field of the current packet.
control_valid	out	1	Qualifies control_out as valid (never asserted when control_out is all-zero).
parity_valid	out	1	The packet's parity checked correct.
parity_not_valid	out	1	The packet's parity checked incorrect.

3 Packet Format

Every packet on input_data has the following structure:



- LENGTH is expressed in **bytes** and equals the size of DATA plus the size of CONTROL and CONTROL_CRC: LENGTH = len(DATA) + 4.
- DATA length must always be a multiple of 32 bits — since LENGTH is expressed in bytes (4 bytes = 32 bits), this means LENGTH itself must always be a multiple of 4.
- PARITY is the running XOR of every 32-bit word of the DATA field.
- If the (corrected) value of CONTROL is zero, it is not driven out on control_out / control_valid.

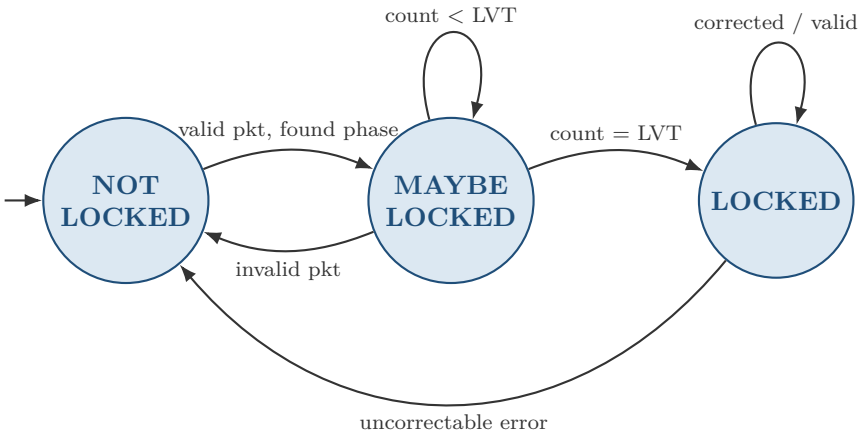
4 Functional Behavior

4.1 Phase Acquisition and Lock

The first bit of a packet is not necessarily the first bit sampled on input_data — a packet may start at any nibble offset within the bus word. On reset, the module does not know this offset (the *phase*) and must search for it.

Search. For every candidate phase (one of INPUT_DATA_WIDTH possible nibble offsets), the module continuously computes the CRC of a 32-bit sliding window and compares it against the LENGTH_CRC that would accompany it. A phase is a candidate match the moment its computed LENGTH_CRC agrees with the field carried in the stream.

Confirmation. The module keeps re-checking the candidate phase on every subsequent packet. Once the *same* phase has produced a valid (LENGTH, LENGTH_CRC) pair for lock_valid_threshold consecutive packets, the phase is considered found and the module locks onto it permanently (until reset). Any packet received while the phase is still being searched for is dropped. A single invalid packet while confirming resets the consecutive count and returns the search to NOT_LOCKED.



4.2 Single-Bit Error Correction

Once `LOCKED`, the module accepts and corrects packets carrying *at most one* bit error in `LENGTH/LENGTH_CRC` **or** in `CONTROL/CONTROL_CRC`: the syndrome of the 32-bit window is matched against the field's single-bit-error signature, the affected bit is flipped, and the corrected value is used downstream. Correction is only attempted after lock; before lock, any CRC mismatch simply fails the phase-search check described above.

4.3 Parity Validation

For every packet, the module accumulates the XOR of all 32-bit `DATA` words and compares it against the trailing 32-bit `PARITY` field, raising exactly one of `parity_valid` / `parity_not_valid`. Parity is a pure indication — it does not gate whether `output_data` is driven.

4.4 Output Control Gating

`control_out` / `control_valid` are only asserted when the (possibly corrected) `CONTROL` value is non-zero; an all-zero `CONTROL` field is treated as "no control for this packet" and is never presented on the output bus.

5 CRC Specification

5.1 Data Window Format

Both CRC checks operate on a 32-bit window:

$$\texttt{window[31:16]} = \text{data_field}, \qquad \texttt{window[15:0]} = \text{crc_field}.$$

The CRC is computed over the 16-bit `data_field`, MSB-first (from bit 15 down to bit 0), and compared against `crc_field`. Both fields share the generic engine `calculate_crc16_generic(data, polynomial, init_val)`.

5.2 Length CRC — CRC-16-CCITT

Polynomial	`0x1021`
Initial value	`0xFFFF`

5.3 Control CRC — CRC-16-IBM/ANSI

Polynomial	`0x8005`
Initial value	`0x0000`

6 Throughput Requirement

The module shall sustain **1 Gbit/s** of incoming payload data on a 4-bit input bus clocked at **1 GHz**.

7 System-Level Assumptions

The following are preconditions the module relies on for correct operation — they are not behaviors the RTL derives on its own, but constraints the surrounding system and traffic generator must uphold.

SYS-01 — Payload alignment

Data payload length is always a multiple of 4 bytes.
Rationale: parity is a 32-bit word and the output data bus is 32 bits wide.

SYS-02 — Lock retention

Once the module reaches `LOCKED`, it is assumed to never need to drop out of lock.
Rationale: after lock, at most one bit error is assumed in `LENGTH` + `LENGTH_CRC` per packet; two or more simultaneous errors after lock fall outside the specified operating envelope (the RTL will fall back to `NOT_LOCKED` and re-search in that case, but this is not a scenario the module is required to handle gracefully).

SYS-03 — Static lock threshold

`lock_valid_threshold` (LVT) is configured once, at the start of operation, and never changes for the lifetime of the lock.
Legal range: $2 \leq LVT \leq 12$. *The port itself is 16 bits wide.*

SYS-04 — Maximum payload size

Data payload length is at most 8 KB.
The maximum payload is 8192 bytes; the `LENGTH` field is 16 bits wide (payload alone does not use its full numeric range).

Note

SYS-02 is an operating assumption, not a hard guarantee: the lock FSM in `dma_rx_fsm.sv` does return to `NOT_LOCKED` on an uncorrectable `LENGTH` CRC failure while locked. The assumption states that, under normal (single-bit-error) conditions, this path is not expected to trigger.